

Programmazione e Tecnologie Web

Dispense del Corso - Lezione 1

Docente: Giuseppe Capasso

Cloud Native Development & Cybersecurity Specialist
Istituto Tecnico Superiore (ITS)

Anno Accademico 2025/2026

Indice

Introduzione al Corso e Strumenti di Sviluppo	2
0.1 Obiettivi del Corso	2
0.2 Obiettivi della Lezione	2
1 L'Architettura del Web e il Modello Client-Server	3
1.1 Il Modello Architetturale Client-Server	3
1.2 Localizzare le Risorse: Indirizzi IP e URL	3
1.3 Il Sistema DNS (Domain Name System)	4
1.4 Il Protocollo HTTP (HyperText Transfer Protocol)	4
1.4.1 Anatomia di una Richiesta HTTP	5
1.4.2 Anatomia di una Risposta HTTP	5
1.5 Interazione con il Browser: API Sincrone vs Asincrone	6
1.5.1 API Sincrone (Blocking)	6
1.5.2 API Asincrone (Non-blocking)	6
1.6 L'Evoluzione del Protocollo HTTP	7
1.6.1 HTTP/0.9 (1991) - The One-Line Protocol	7
1.6.2 HTTP/1.0 (1996) - L'Espansione	7
1.6.3 HTTP/1.1 (1997) - L'Ottimizzazione	8
2 Setup dell'Ambiente di Lavoro	10
2.1 L'Editor di Codice: Visual Studio Code	10
2.1.1 Estensioni Consigliate	10
2.2 Il Controllo di Versione: Git	10
2.2.1 Installazione di Git	10
2.2.2 Git: Un Sistema Distribuito	11
2.2.3 Il Workflow di Git: Concetti Base ed Esempi	11
2.2.4 Risorse per Approfondire	12
3 Laboratorio: Primi Passi nel Frontend	13
3.1 Esercizi Guidati	13
3.1.1 Esercizio 1: Il browser come interprete	13
3.1.2 Esercizio 2: Manipolazione del DOM	13
3.1.3 Esercizio 3: Gestione degli Eventi	13
3.1.4 Esercizio 4: Form Modulare	14
3.1.5 Esercizio 5: Gestione delle liste	14

Introduzione al Corso e Strumenti di Sviluppo

0.1 Obiettivi del Corso

Oltre ai singoli traguardi di ogni lezione, il corso si pone l'obiettivo di portarvi a costruire progetti concreti che simulino scenari reali di sviluppo web:

- **Todo Application:** Svilupperemo una completa applicazione per la gestione dei compiti. Questo sarà il nostro banco di prova per imparare il paradigma **CRUD** (Create, Read, Update, Delete), ovvero le quattro operazioni fondamentali di ogni software che gestisce dati.
- **Chat Application:** Concluderemo il percorso realizzando una piccola applicazione di chat in tempo reale. Questo ci permetterà di esplorare tecnologie avanzate che vanno oltre il semplice modello richiesta-risposta, affrontando il tema della comunicazione bidirezionale.

0.2 Obiettivi della Lezione

Lo scopo di questo corso è fornire le competenze necessarie per comprendere, progettare e sviluppare applicazioni web complete (end-to-end). Il livello di partenza assume una conoscenza basilare della logica di programmazione, ma esploreremo da zero l'ecosistema web.

Oggi, un'applicazione web non è più una semplice vetrina di documenti testuali (come era negli anni '90), ma un software complesso e interattivo distribuito su una rete. In questo primo capitolo, definiremo il perimetro di azione di uno sviluppatore e inizieremo il nostro percorso.

Capitolo 1

L'Architettura del Web e il Modello Client-Server

1.1 Il Modello Architetture Client-Server

Internet è un'enorme rete di calcolatori interconnessi. Il Web, che è uno dei servizi che "girano" su Internet, si basa quasi interamente sull'architettura **Client-Server**. Si tratta di un modello distribuito in cui i ruoli sono asimmetrici (si veda la [Figura 1.1](#)):

- **Il Client:** È il dispositivo (o meglio, il software su quel dispositivo) che *richiede* una risorsa o un servizio. Nel nostro contesto, il client per eccellenza è il **Browser web** (Chrome, Firefox, Safari). Il client è la parte "attiva" della comunicazione: avvia sempre la conversazione.
- **Il Server:** È un computer (o un programma in esecuzione su di esso) sempre acceso e in ascolto, in attesa di ricevere richieste. Quando ne riceve una, la elabora e fornisce una *risposta*. Server web famosi sono Apache, Nginx o applicazioni scritte in Express/Flask.

La regola fondamentale è: **Il server non invia mai dati al client senza che questi siano stati esplicitamente richiesti.**

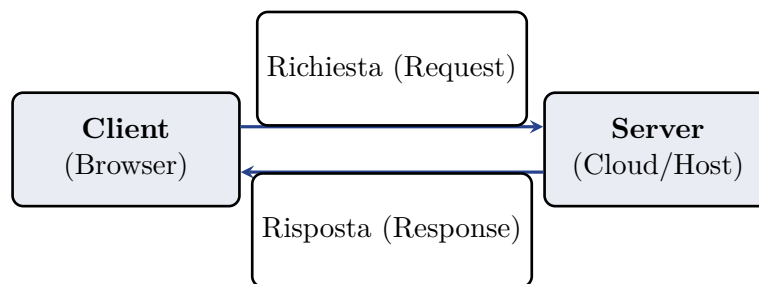


Figura 1.1: Modello architetturale Client-Server.

1.2 Localizzare le Risorse: Indirizzi IP e URL

Affinché il client possa contattare il server corretto su scala globale, è necessario un sistema di indirizzamento. I server (così come i client) sono identificati da un **Indirizzo IP** (es. 142.250.184.46). Tuttavia, gli IP sono difficili da memorizzare per gli esseri umani. Per questo utilizziamo le URL.

Un **URL** (Uniform Resource Locator) è una stringa che definisce in modo univoco dove si trova una risorsa e come accedervi. La struttura formale di un URL è la seguente:

```
schema://host:porta/percorso?query_string#frammento
```

- **schema (scheme):** Indica il protocollo da utilizzare (es. `http`, `https`, `ftp`).
- **host:** Il nome a dominio del server (es. `www.esempio.com`) o l'indirizzo IP.

- **porta (port):** Un numero che identifica il processo logico sul server. Se omesso, si assumono le porte standard (80 per HTTP, 443 per HTTPS).
- **percorso (path):** La posizione gerarchica della risorsa specifica all'interno del server (es. `/utenti/profilo.html`).
- **query string:** Parametri opzionali passati alla risorsa, introdotti dal punto interrogativo `?` e separati dalla e-commerce `&` (es. `?id=123&lang=it`).
- **frammento (fragment):** Identifica una sezione specifica all'interno della risorsa, prelevata dall'hash `#` (es. `#paragrafo2`). Non viene inviato al server, è gestito dal browser.

1.3 Il Sistema DNS (Domain Name System)

Poiché la rete instrada le comunicazioni utilizzando gli indirizzi IP numerici, ma gli utenti utilizzano nomi a dominio testuali (URL), serve un sistema di traduzione.

Il **DNS** agisce come la rubrica telefonica di Internet. Quando digiti `www.google.com` nel browser, succede quanto segue:

1. Il browser chiede al sistema operativo se conosce già l'IP di quel dominio (cache locale).
2. In caso negativo, il sistema interroga un **Server DNS** (solitamente fornito dal provider di rete, o servizi pubblici come 8.8.8.8 di Google).
3. Il Server DNS cerca nei suoi registri il nome a dominio e restituisce al browser il corrispondente Indirizzo IP.
4. Solo a questo punto il browser può instaurare una connessione diretta con il server.

1.4 Il Protocollo HTTP (HyperText Transfer Protocol)

Una volta instaurata la connessione (tipicamente tramite TCP/IP), client e server devono comunicare utilizzando una lingua comune. Questo linguaggio è il protocollo **HTTP**.

L'HTTP è un protocollo **stateless** (senza stato): ogni richiesta è totalmente indipendente dalle precedenti e dalle successive. Il server non ha memoria di default delle interazioni passate con un client (problema che risolveremo in seguito usando i Cookie e i Token). L'interazione HTTP è divisa strettamente in due fasi: **Richiesta** (inviata dal client) e **Risposta** (restituita dal server), come illustrato nella [Figura 1.2](#). L'HTTP è un protocollo basato su testo in chiaro.

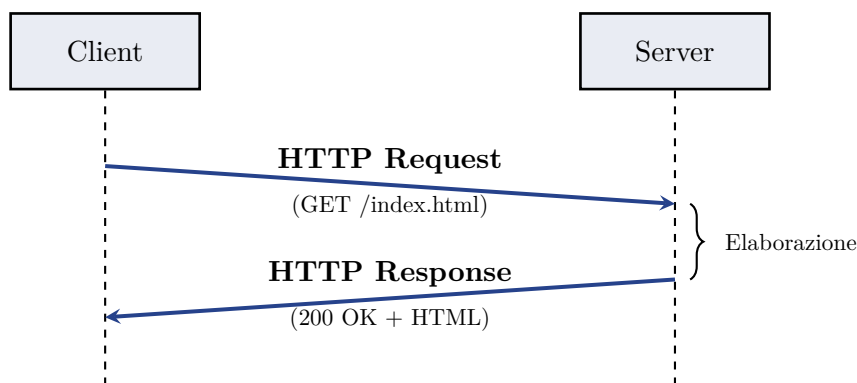


Figura 1.2: Ciclo di Richiesta e Risposta HTTP.

1.4.1 Anatomia di una Richiesta HTTP

Una richiesta HTTP è composta da una riga iniziale, delle intestazioni (Headers) e un corpo opzionale (Body).

```
GET /index.html?lang=it HTTP/1.1
Host: www.esempio.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html
Accept-Language: it-IT
```

Listing 1.1: Esempio di Richiesta HTTP cruda

La riga iniziale definisce:

- **Metodo HTTP (Verbo):** Indica l'intenzione dell'operazione. I principali sono:
 - **GET:** Richiede il recupero di una risorsa (sola lettura).
 - **POST:** Invia dati al server per essere elaborati o salvati (es. invio di un form).
 - **PUT:** Aggiorna o crea una risorsa sostituendola integralmente.
 - **DELETE:** Richiede l'eliminazione di una risorsa.
- **Path:** Il percorso della risorsa richiesta (`/index.html`).
- **Versione:** Es. HTTP/1.1 o HTTP/2.

Gli **Headers** (`Host`, `User-Agent`) sono coppie chiave-valore che passano metadati aggiuntivi al server.

1.4.2 Anatomia di una Risposta HTTP

Il server risponde con un blocco di testo simile, che include lo stato dell'operazione e i dati richiesti.

```
HTTP/1.1 200 OK
Date: Mon, 01 Jan 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 138

<!DOCTYPE html>
<html>
  <head><title>Benvenuto</title></head>
  <body><h1>Ciao Mondo!</h1></body>
</html>
```

Listing 1.2: Esempio di Risposta HTTP cruda

La riga iniziale della risposta contiene lo **Status Code** (Codice di Stato), fondamentale per capire l'esito della richiesta:

- **1xx (Informativi):** Richiesta ricevuta, elaborazione in corso.
- **2xx (Successo):** L'azione è stata ricevuta, compresa e accettata con successo (es. 200 OK, 201 Created).
- **3xx (Reindirizzamento):** È necessaria un'ulteriore azione da parte del client (es. 301 Moved Permanently).

- **4xx (Errore del Client):** La richiesta contiene una sintassi errata o non può essere soddisfatta. È colpa del browser o dell'utente (es. 400 Bad Request, 403 Forbidden, 404 Not Found).
- **5xx (Errore del Server):** Il server ha fallito nel soddisfare una richiesta apparentemente valida, a causa di un bug o di un crollo interno (es. 500 Internal Server Error, 502 Bad Gateway).

Dopo gli headers della risposta, separato da una riga vuota, si trova il **Body** (corpo), che contiene l'effettivo documento HTML, l'immagine o l'oggetto JSON richiesto.

1.5 Interazione con il Browser: API Sincrone vs Asincrone

Lo sviluppo web moderno non riguarda solo la visualizzazione di documenti, ma l'interazione dinamica con l'utente e con i dati. Il browser mette a disposizione degli sviluppatori una serie di strumenti (API) per manipolare la pagina e comunicare con l'esterno. La distinzione più importante in questo contesto è tra operazioni **Sincrone** e **Asincrone**.

1.5.1 API Sincrone (Blocking)

In un modello sincrono, le operazioni vengono eseguite una dopo l'altra. Se un'operazione richiede tempo (es. un calcolo complesso o l'attesa di un input), il "filo" dell'esecuzione (Main Thread) si ferma. Poiché il browser usa un unico thread per gestire sia il codice JavaScript che il rendering della pagina (grafica), un'operazione sincrona lenta **blocca l'interfaccia utente**.

- **Esempio classico:** La funzione `alert()` di JavaScript. Finché l'utente non preme "OK", il browser è totalmente congelato: non puoi scorrere la pagina, cliccare pulsanti o vedere animazioni.
- **Vantaggi:** Logica semplice da seguire (flusso lineare).
- **Svantaggi:** Pessima esperienza utente (il sito sembra "rotto" o bloccato).

1.5.2 API Asincrone (Non-blocking)

Le API asincrone permettono di avviare un'operazione e "mettersi in attesa" della sua conclusione senza bloccare il resto del programma. Il browser continua a rispondere ai clic dell'utente e a renderizzare la pagina, mentre l'operazione lenta (es. scaricare un'immagine o i dati di una Todo list) prosegue in background.

- **Tecnologie chiave:** **AJAX** (Asynchronous JavaScript and XML) e la moderna API **Fetch**.
- **Vantaggi:** Interfaccia sempre fluida e reattiva; possibilità di aggiornare solo parti della pagina senza ricaricarla interamente.
- **Svantaggi:** Codice più complesso da gestire (necessità di usare Callback, Promises o `async/await`).

Le differenze tra questi due approcci sono visualizzate nella [Figura 1.3](#) e nella [Figura 1.4](#).

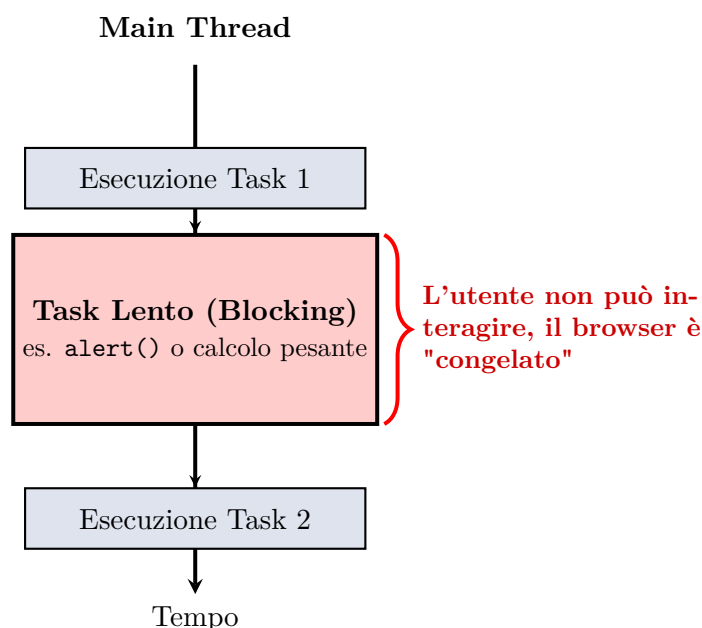


Figura 1.3: Modello di esecuzione sincrono (bloccante).

1.6 L'Evoluzione del Protocollo HTTP

Il protocollo HTTP non è rimasto statico nel tempo. Dalla sua nascita al CERN di Ginevra per mano di Tim Berners-Lee, si è evoluto per gestire pagine sempre più ricche di risorse (immagini, script, stili) e per ridurre i tempi di caricamento, che sono critici per l'esperienza utente.

1.6.1 HTTP/0.9 (1991) - The One-Line Protocol

Nato per scambiare semplici documenti testuali, HTTP/0.9 era un protocollo estremamente minimale. Non esisteva ancora il concetto di "multimedialità" nel web come lo intendiamo oggi; l'obiettivo era semplicemente recuperare un file di testo.

- **Semplicità assoluta:** La richiesta era composta da una sola riga: `GET /file.html`.
- **Senza Metadati:** Non esistevano intestazioni (headers), quindi il server non poteva inviare informazioni extra come il tipo di file o la data.
- **Solo HTML:** Il protocollo poteva trasferire solo documenti HTML.

1.6.2 HTTP/1.0 (1996) - L'Espansione

Con la crescita del Web, divenne necessario trasmettere non solo testo ma anche immagini e altri tipi di file. HTTP/1.0 introdusse la struttura a "messaggio" che usiamo ancora oggi, con intestazioni e codici di stato. Inoltre, soffriva di un grave problema di performance: ogni risorsa richiedeva una nuova connessione TCP (come mostrato nella [Figura 1.5](#)).

- **Introduzione degli Headers:** Permise di negoziare il tipo di contenuto (*Content-Type*) e altre impostazioni.
- **Codici di Stato:** Fondamentali per far capire al browser se la richiesta era andata a buon fine o meno (es. 404).

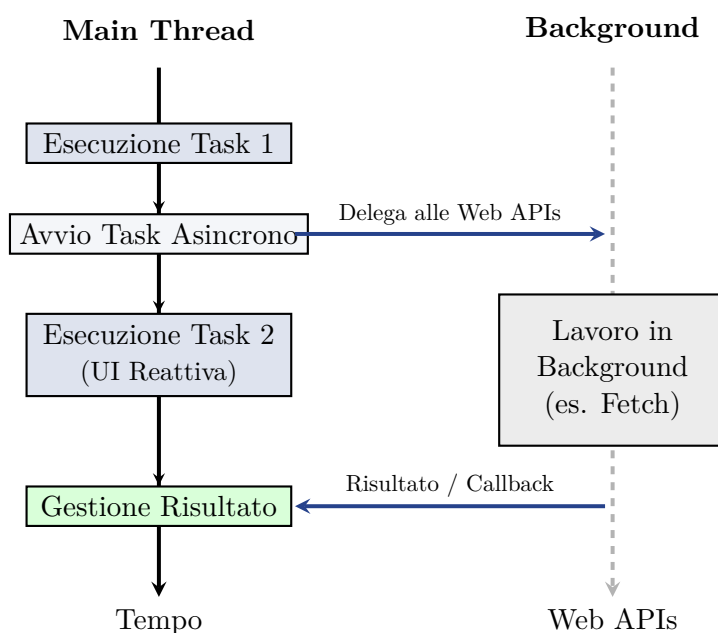


Figura 1.4: Modello di esecuzione asincrono (non-bloccante).

1.6.3 HTTP/1.1 (1997) - L'Ottimizzazione

HTTP/1.1 è diventato lo standard de facto del Web per oltre un decennio. Il suo contributo principale è stato il concetto di **connessione persistente**, che permette di abbattere drasticamente la latenza evitando i ripetuti "handshake" TCP (si veda la [Figura 1.6](#)).

- **Keep-Alive:** La connessione TCP rimane aperta dopo la prima risposta, permettendo al client di inviare immediatamente altre richieste sulla stessa "linea".
- **Virtual Hosting:** Grazie all'header `Host`, un server può distinguere tra più siti web ospitati sullo stesso indirizzo IP.
- **Caching Migliorato:** Introduzione di controlli più granulari su quanto tempo una risorsa può rimanere nella memoria del browser.

Nelle prossime lezioni vedremo come l'evoluzione sia continuata. Tuttavia, è importante sottolineare che **la stragrande maggioranza delle applicazioni statiche e dei siti web moderni funziona perfettamente con HTTP/1.1**. Sebbene esistano versioni più recenti, la 1.1 rimane una solida base per comprendere il web.

Verso la fine del corso, quando avremo padroneggiato le basi, analizzeremo quali sono i limiti strutturali che hanno portato alla nascita di **HTTP/2** e **HTTP/3** e come queste tecnologie risolvano problemi di latenza estrema e multiplexing delle risorse.

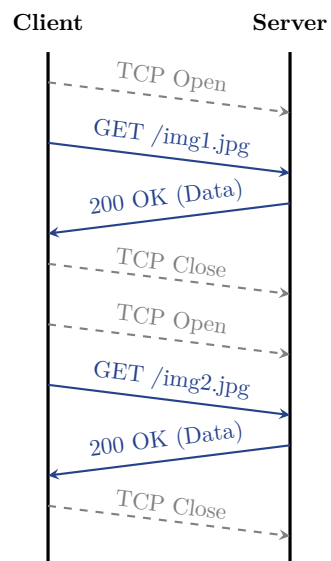


Figura 1.5: Interazione HTTP/1.0 con connessioni TCP multiple.

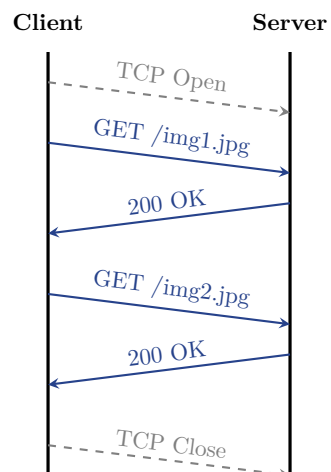


Figura 1.6: Interazione HTTP/1.1 con connessioni persistenti (Keep-Alive).

Capitolo 2

Setup dell'Ambiente di Lavoro

Per sviluppare software in modo professionale, non possiamo limitarci a usare un semplice editor di testo. Abbiamo bisogno di strumenti che ci aiutino a scrivere codice più velocemente, a individuare errori e a gestire le diverse versioni del nostro lavoro.

2.1 L'Editor di Codice: Visual Studio Code

Visual Studio Code (spesso chiamato **VS Code**) è attualmente lo standard de facto per lo sviluppo web. È un editor gratuito, open-source e leggerissimo, ma estremamente potente grazie al suo sistema di estensioni.

- **Download:** Potete scaricarlo dal sito ufficiale: <https://code.visualstudio.com/>.
- **Perché VS Code?** Offre evidenziazione della sintassi, completamento automatico intelligente (IntelliSense) e un terminale integrato che useremo moltissimo.

2.1.1 Estensioni Consigliate

Le estensioni permettono di aggiungere funzionalità a VS Code. Durante il corso useremo principalmente:

- **Italian Language Pack:** Per tradurre l'interfaccia in italiano.
- **Prettier - Code formatter:** Per formattare automaticamente il codice e renderlo ordinato.
- **Live Server:** Fondamentale per questa lezione. Crea un piccolo server locale che aggiorna automaticamente il browser ogni volta che salvate un file HTML o CSS. **Installatela subito!**

2.2 Il Controllo di Versione: Git

2.2.1 Installazione di Git

Prima di poter utilizzare Git, è necessario installarlo sul proprio computer:

- **Windows:** Scaricate il programma di installazione da <https://git-scm.com/>. Seguite il wizard di installazione accettando le impostazioni predefinite.
- **macOS:** Se avete installato i command line tools di Xcode, Git potrebbe essere già presente. Altrimenti, scaricatelo dal sito ufficiale o usate Homebrew: `brew install git`.
- **Linux:** Solitamente è preinstallato. In caso contrario, usate il gestore pacchetti della vostra distribuzione (es. `sudo apt install git` su Ubuntu).

Dopo l'installazione, verificate il funzionamento aprendo il terminale e scrivendo: `git -version`.

Il controllo di versione è la capacità di tenere traccia di ogni singola modifica apportata al codice. Immaginate Git come una "macchina del tempo" per il vostro progetto. Lo impareremo a usare gradualmente durante tutto il corso.

2.2.2 Git: Un Sistema Distribuito

Git è un sistema di controllo di versione **distribuito**. A differenza di sistemi centralizzati, dove ogni modifica deve essere inviata subito al server, in Git ogni sviluppatore ha una copia completa dell'intero storico del progetto sul proprio computer.

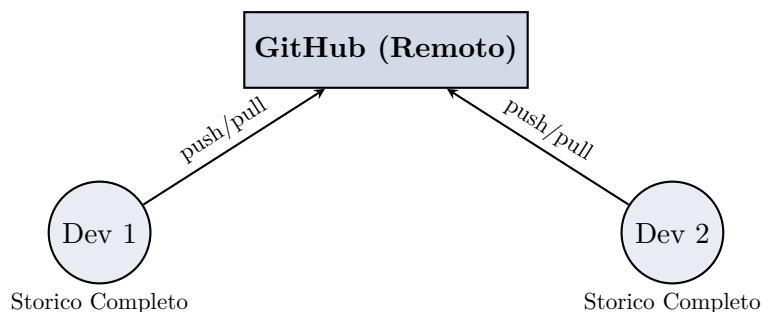


Figura 2.1: Git è distribuito: ogni sviluppatore possiede l'intero database locale.

Branch Un *branch* (ramo) è essenzialmente un puntatore a un determinato commit. I branch permettono di lavorare su nuove funzionalità o correggere bug in isolamento, senza compromettere il codice stabile (il branch principale, solitamente chiamato **main** o **master**).

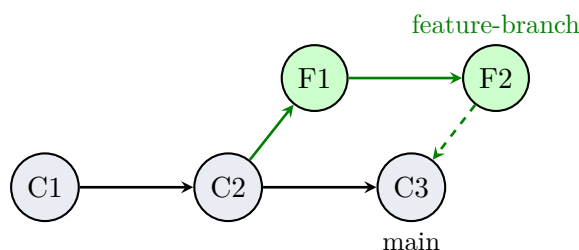


Figura 2.2: Lavoro parallelo con i branch: la funzionalità viene sviluppata separatamente e poi integrata nel branch main.

2.2.3 Il Workflow di Git: Concetti Base ed Esempi

Per capire come funziona Git, dobbiamo immaginare che il nostro progetto passi attraverso tre stati principali:

1. **Working Directory:** La cartella dove state modificando i file.
2. **Staging Area:** Una "sala d'attesa" dove preparate i file che volete includere nel prossimo salvataggio.
3. **Repository (Git Directory):** Il punto in cui i file vengono salvati in modo permanente (i *commit*).

Ecco un esempio pratico di un flusso di lavoro tipico:

1. **Inizializzazione:** Entrate nella cartella del vostro progetto dal terminale e scrivete:

```
$ git init
```

Questo comando crea una cartella nascosta `.git` che contiene tutto lo storico del progetto.

2. **Verifica dello stato:** Per vedere quali file avete modificato, usate:

```
$ git status
```

Git vi indicherà in rosso i file che sono stati modificati ma non ancora "preparati".

3. **Staging:** Per aggiungere un file alla sala d'attesa (Staging Area), usate:

```
$ git add index.html  
$ git add .
```

Il primo comando aggiunge un singolo file; il secondo aggiunge tutti i file modificati nella cartella corrente.

4. **Commit:** Una volta pronti, salvate le modifiche nel repository:

```
$ git commit -m "Aggiunta struttura base stile CSS"
```

Il messaggio (indicato da `-m`) è fondamentale: deve descrivere brevemente cosa avete cambiato, così da poter ritrovare facilmente la versione corretta in futuro.

2.2.4 Risorse per Approfondire

Se volete portarvi avanti o consultare materiale aggiuntivo, ecco alcuni tutorial eccellenti:

- **Pro Git Book:** La guida ufficiale e completa (<https://git-scm.com/book/it/v2>).
- **Oh My Git!:** Un gioco interattivo per imparare Git (<https://ohmygit.org/>).
- **Git Immersion:** Un tour guidato nei fondamenti di Git (<https://gitimmersion.com/>).

Capitolo 3

Laboratorio: Primi Passi nel Frontend

Ora che conosciamo l'architettura del web, iniziamo a scrivere codice. Per questo laboratorio, useremo **Visual Studio Code (VS Code)** come nostro strumento principale.

3.1 Esercizi Guidati

Il materiale per questi esercizi si trova nella cartella `src/lezione1/` del repository.

3.1.1 Esercizio 1: Il browser come interprete

Un file HTML è un documento dichiarativo. Quando lo apriamo, il browser lo analizza (parse) costruendo una struttura in memoria. Lo script incluso viene eseguito nel contesto della pagina.

```
<!DOCTYPE html>
<html>
<body>
  <h1>Ciao Mondo</h1>
  <script>console.log("Browser attivo!");</script>
</body>
</html>
```

Il browser espone l'oggetto globale `window` e l'oggetto `document`, che rappresenta l'intera pagina.

Task: Cambia il colore di sfondo della pagina scrivendo `document.body.style.backgroundColor = 'yellow'`; nello script.

3.1.2 Esercizio 2: Manipolazione del DOM

Il **DOM (Document Object Model)** è l'interfaccia che permette a JavaScript di agire sulla pagina. Ogni tag HTML diventa un oggetto JavaScript. `getElementById` è il metodo base per cercare un nodo specifico nell'albero.

```
<div id="container">
  <p id="first">Primo</p>
</div>
<script>
  const container = document.getElementById('container');
  container.style.backgroundColor = "lightblue";
</script>
```

Task: Modifica il testo del paragrafo `first` usando `document.getElementById('first').innerText = 'Testo modificato'`;

3.1.3 Esercizio 3: Gestione degli Eventi

Il browser è un ambiente *event-driven*. Un evento è un segnale di notifica. `addEventListener` associa una funzione (callback) a un tipo di evento (click, mouseover, keydown).

```
<button id="myBtn">Cliccami!</button>
<p id="msg"></p>
<script>
  const btn = document.getElementById('myBtn');
  btn.addEventListener('click', () => {
    document.getElementById('msg').innerText = "Hai cliccato!";
  });
</script>
```

Task: Aggiungi un ascoltatore per l'evento `mouseover` sul bottone che cambia il colore del testo.

3.1.4 Esercizio 4: Form Modulare

Per progetti complessi, dividiamo i file. Il browser scarica i file referenziati.

```
<form id="myForm">
  <input type="email" required placeholder="Email">
  <button type="submit">Invia</button>
</form>
<script src="index.js"></script>
```

Listing 3.1: index.html

L'evento `submit` va intercettato chiamando `e.preventDefault()` per evitare che il browser ricarichi la pagina. **Task:** Nel file `index.js`, aggiungi un campo `number` per l'età nel form e mostralo nell'alert al submit.

3.1.5 Esercizio 5: Gestione delle liste

Possiamo creare nodi HTML dinamicamente. `createElement` crea un nuovo oggetto DOM non ancora visibile, `appendChild` lo inserisce come ultimo figlio di un genitore.

```
<ul id="lista"><li>Pane</li></ul>
<button id="addBtn">Aggiungi</button>
<script>
  document.getElementById('addBtn').addEventListener('click', ()
    => {
    const li = document.createElement('li');
    li.innerText = 'Nuovo elemento';
    document.getElementById('lista').appendChild(li);
  });
</script>
```

Task: Aggiungi un bottone "Rimuovi" che elimina l'ultimo elemento della lista usando `lista.lastElementChild`.